



Documentación - Encriptador RALABE



Descripción General

Encriptador RALABE es una aplicación de escritorio desarrollada en C# con .NET 10, diseñada para encriptar y desencriptar archivos SQL de forma segura. La aplicación utiliza algoritmos criptográficos modernos para proteger la información sensible de bases de datos.



Propósito

El aplicativo permite:

- **Encriptar** archivos SQL (`.sql`) a un formato protegido (`.ralabe`)
 - **Desencriptar** archivos protegidos (`.ralabe`) de vuelta a SQL (`.sql`)
 - Gestionar archivos de base de datos con contraseña segura
 - Proteger copias de seguridad y respaldos de información crítica
-



Características de Seguridad

Algoritmo de Encriptación

- **Cifrado:** AES-256 (Advanced Encryption Standard)
- **Modo:** CBC (Cipher Block Chaining)
- **Padding:** PKCS7

Derivación de Clave

- **Algoritmo:** PBKDF2 (Password-Based Key Derivation Function 2)
- **Función Hash:** SHA-256
- **Iteraciones:** 10,000
- **Salt:** 32 bytes generados aleatoriamente para cada encriptación

Manejo de Archivos

- **FileStream:** Procesa archivos en bloques de 8KB
 - **Ventaja:** Manejo eficiente de archivos grandes sin saturar la memoria RAM
 - **Escalabilidad:** Soporta archivos de cualquier tamaño
-



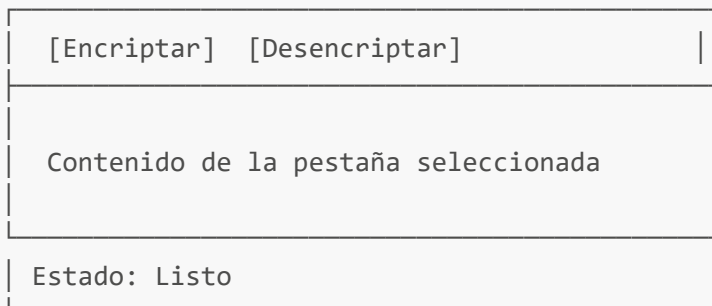
Requisitos del Sistema

- **Framework:** .NET 10
 - **Sistema Operativo:** Windows
 - **Memoria RAM:** Mínimo 512MB (recomendado 2GB o más)
 - **Espacio Disco:** Suficiente para los archivos a procesar
-

Interfaz Gráfica

Estructura Principal

La aplicación cuenta con una interfaz basada en **TabControl** con dos pestañas principales:



Pestaña 1: Encriptar

Propósito: Convertir archivos SQL a formato encriptado RALABE

Componentes:

- **Label:** "Archivo SQL:"
- **TextBox:** Muestra la ruta del archivo seleccionado (solo lectura)
- **Botón "Seleccionar":** Abre un diálogo para elegir archivo **.sql**
- **Botón "Procesar (Encriptar)":** Inicia el proceso de encriptación

Flujo de Uso:

1. Haz clic en "Seleccionar" para elegir tu archivo **.sql**
2. La ruta aparecerá en el TextBox
3. Haz clic en "Procesar (Encriptar)"
4. Se abrirá un diálogo para guardar el archivo encriptado
5. Elige la ubicación y nombre (sugerencia: **nombre.ralabe**)
6. El archivo se encriptará y se mostrará un mensaje de éxito
7. El TextBox se limpiará automáticamente

Pestaña 2: Desencriptar

Propósito: Convertir archivos RALABE encriptados de vuelta a SQL

Componentes:

- **Label:** "Archivo .RALABE:"
- **TextBox:** Muestra la ruta del archivo seleccionado (solo lectura)
- **Botón "Seleccionar":** Abre un diálogo para elegir archivo **.ralabe**
- **Botón "Restaurar a SQL":** Inicia el proceso de desencriptación

Flujo de Uso:

1. Haz clic en "Seleccionar" para elegir tu archivo `.ralabe`
2. La ruta aparecerá en el TextBox
3. Haz clic en "Restaurar a SQL"
4. Se abrirá un diálogo para guardar el archivo descriptado
5. Elige la ubicación y nombre (sugerencia: `nombre.sql`)
6. El archivo se descriptará y se mostrará un mensaje de éxito
7. El TextBox se limpiará automáticamente

Barra de Estado

Ubicada en la parte inferior de la ventana, muestra:

- **"Listo"**: Esperando acción del usuario
- **"Encriptando archivo..."**: Proceso en progreso
- **"Desencriptando archivo..."**: Proceso en progreso
- **Mensajes de éxito o error**: Confirmación final de la operación

Componentes Principales

1. Clase `FileSecurity`

Ubicación: `FileSecurity.cs`

Descripción: Clase estática que contiene toda la lógica criptográfica.

Métodos Públicos:

`EncryptFile(string inputPath, string outputPath, string password)`

- **Parámetros:**
 - `inputPath`: Ruta del archivo a encriptar
 - `outputPath`: Ruta donde guardar el archivo encriptado
 - `password`: Contraseña para proteger el archivo
- **Proceso:**
 1. Valida que el archivo de entrada existe
 2. Genera un salt aleatorio de 32 bytes
 3. Deriva la clave AES usando PBKDF2
 4. Encripta el archivo en bloques usando FileStream
 5. Almacena el salt al inicio del archivo de salida
- **Excepciones:** Lanza `FileNotFoundException` o `InvalidOperationException` en caso de error

`DecryptFile(string inputPath, string outputPath, string password)`

- **Parámetros:**
 - `inputPath`: Ruta del archivo encriptado a desencriptar
 - `outputPath`: Ruta donde guardar el archivo desencriptado
 - `password`: Contraseña para desproteger el archivo
- **Proceso:**

1. Valida que el archivo de entrada existe
2. Lee el salt del inicio del archivo encriptado
3. Deriva la clave AES usando PBKDF2 con el mismo salt
4. Descripta el archivo en bloques usando FileStream
5. Guarda el contenido original en el archivo de salida

- **Excepciones:** Lanza `FileNotFoundException` o `CryptographicException` si la contraseña es incorrecta

Características de Seguridad Internas:

```
private const int SaltSize = 32;           // 32 bytes de salt
private const int KeySize = 32;           // 256 bits para AES-256
private const int IvSize = 16;            // 128 bits para IV
private const int Iterations = 10000;     // Iteraciones PBKDF2
private const int BufferSize = 8192;      // 8KB por lectura
```

2. Formulario Principal `Form1`

Ubicación: `Form1.cs` y `Form1.Designer.cs`

Descripción: Interfaz gráfica principal de la aplicación.

Controles:

- TabControl con 2 pestañas
- TextBox de solo lectura para rutas
- OpenFileDialog para seleccionar archivos
- SaveFileDialog para elegir ubicación de guardado
- Botones para seleccionar y procesar
- StatusStrip para mostrar estado

Características:

- Validación de rutas antes de procesar
- Deshabilita botones durante el proceso para evitar dobles clics
- Muestra diálogos MessageBox con feedback al usuario
- Limpia TextBox automáticamente después de procesar exitosamente
- Manejo robusto de excepciones



Ejemplo de Uso

Encriptar un archivo SQL

1. Abre la aplicación
2. Asegúrate de estar en la pestaña "Encriptar"
3. Haz clic en "Seleccionar"
4. Navega a: `C:\Bases_Datos\respaldo.sql`

5. Haz clic en "Procesar (Encriptar)"
6. Elige guardar en: C:\Bases_Datos\respaldo.ralabe
7. Espera a que termine
8. ¡Listo! Tu archivo está encriptado

Desencriptar un archivo RALABE

1. Abre la aplicación
2. Ve a la pestaña "Desencriptar"
3. Haz clic en "Seleccionar"
4. Navega a: C:\Bases_Datos\respaldo.ralabe
5. Haz clic en "Restaurar a SQL"
6. Elige guardar en: C:\Bases_Datos\respaldo_restaurado.sql
7. Espera a que termine
8. ¡Listo! Tu archivo está desencriptado

Consideraciones Importantes

Contraseña

- **Contraseña Predeterminada:** `RalabeSecureKey2024`
- Esta contraseña se usa automáticamente en todas las operaciones
- Se recomienda modificarla en el código fuente para mayor seguridad

Backup

- **Siempre mantén un respaldo** de tus archivos originales
- Los archivos encriptados son irrecuperables sin la contraseña correcta

Integridad del Archivo

- El formato RALABE contiene el salt en los primeros 32 bytes
- No modifiques estos bytes o la desencriptación fallará

Archivos Grandes

- Gracias al uso de FileStream, la aplicación puede manejar archivos de cualquier tamaño
- No importa si es 1MB o 10GB, el consumo de RAM será el mismo (~8KB por operación)

Manejo de Errores

Errores Comunes

Error	Causa	Solución
"El archivo no existe"	La ruta especificada no existe	Verifica que la ruta sea correcta

Error	Causa	Solución
"La contraseña puede ser incorrecta"	Contraseña incorrecta al descriptar	Asegúrate de usar la contraseña correcta
"Archivo inválido: salt incorrecto"	El archivo está corrupto o no es RALABE	Verifica que sea un archivo generado por esta app
"Acceso denegado"	Permisos insuficientes	Ejecuta con permisos de administrador

Manejo Automático de Errores

- Los TextBox se limpian parcialmente en caso de error
- Los archivos de salida incompletos se eliminan automáticamente
- Se muestra un mensaje descriptivo en cada error



Rendimiento

Velocidad de Procesamiento

- Depende del tamaño del archivo y la velocidad del disco
- Archivos pequeños (< 100MB): Generalmente < 1 segundo
- Archivos medianos (100MB - 1GB): 1 - 5 segundos
- Archivos grandes (> 1GB): 5+ segundos

Optimizaciones

- FileStream con buffer de 8KB para máxima eficiencia
- CryptoStream para procesamiento en streaming
- No carga archivos completos en memoria



Estructura del Proyecto

```
encriptador_ralabe/  
├── Form1.cs           # Lógica de la interfaz  
├── Form1.Designer.cs # Diseño de la interfaz  
├── FileSecurity.cs    # Lógica criptográfica  
├── Program.cs         # Punto de entrada  
├── encriptador_ralabe.csproj # Archivo de proyecto  
└── documentación/  
    └── documentacion.md # Este archivo
```



Soporte y Contacto

Desarrollador: RALABE **Versión:** 1.0 **Framework:** .NET 10 **Lenguaje:** C#

Licencia

Este proyecto es software propietario. Todos los derechos reservados.

Conceptos Técnicos

¿Por qué AES-256?

- Es el estándar de encriptación más seguro actualmente
- Usado por gobiernos y empresas Fortune 500
- Resistente a ataques de fuerza bruta

¿Por qué PBKDF2?

- Convierte contraseña en clave criptográfica segura
- Las 10,000 iteraciones ralentizan ataques de fuerza bruta
- Estándar en la industria (RFC 2898)

¿Por qué el salt?

- Previene ataques con tablas precalculadas (rainbow tables)
- 32 bytes = 256 bits de aleatoriedad
- Cada encriptación es única incluso con la misma contraseña

¿Por qué FileStream?

- Permite procesar archivos sin cargarlos completamente en RAM
- Ideal para archivos grandes
- Consume memoria constante independientemente del tamaño

Documento generado: 2024 **Última actualización:** 2024